Guide détaillé

ROS 2 + Webots : comment on travaille, et le lien avec AUTOSAR

Projet Lighting System Control — Amina Borcheni — M2 Ingénierie Automobile et Aéronautique, ENET'Com — MATLAB R2022b, ROS 2 Lyrical, Webots R2025a, Ubuntu 26.04 (WSL 2)

Ce guide répond à trois questions : (1) comment fonctionnent ROS 2 et Webots et comment on les utilise au quotidien ; (2) comment notre calculateur d'éclairage a été branché dessus ; (3) comment tout cela se relie à l'architecture **AUTOSAR** définie dans le cahier des charges (SWC, ports, runnables, RTE, BSW).



La démonstration : la Tesla virtuelle de nuit, feux de route commandés par le code C du calculateur via ROS 2

Chapitre	Contenu
1	Le principe en une page
2	ROS 2 : les notions indispensables et les commandes utiles
3	Webots : monde, robots, capteurs, superviseur, lien avec ROS 2
4	Notre projet : paquets, nœuds, topics, cycle de 10 ms
5	Le lien avec AUTOSAR : SWC, ports, runnables, RTE, BSW
6	La démonstration (captures et 15 tests)
7	Mode opératoire : modifier, recompiler, relancer
8	Dépannage, limites, questions du jury, glossaire

1. Le principe en une page

Un calculateur automobile ne vit jamais seul : il **reçoit** des signaux (capteur de lumière, vitesse, commodo, caméra), **décide**, puis **commande** des sorties (lampes). Pour le tester sans voiture, il faut lui fournir un environnement qui produit ces signaux et réagit à ses commandes. C'est le rôle de la virtualisation :

Rôle	Dans une vraie voiture	Dans notre virtualisation
Environnement physique	Route, soleil, tunnel, autres véhicules	**Webots** (simulateur 3D)
Réseau de communication	Bus CAN / Ethernet automobile	**ROS 2** (topics, middleware DDS)
Logiciel de base + RTE	Pile AUTOSAR (RTE, COM, Dem, OS)	Nœud **lsc_ecu** (enveloppe C++)
Logique de l'éclairage	SWC AUTOSAR (code C)	**Le même code C**, généré par Embedded Coder

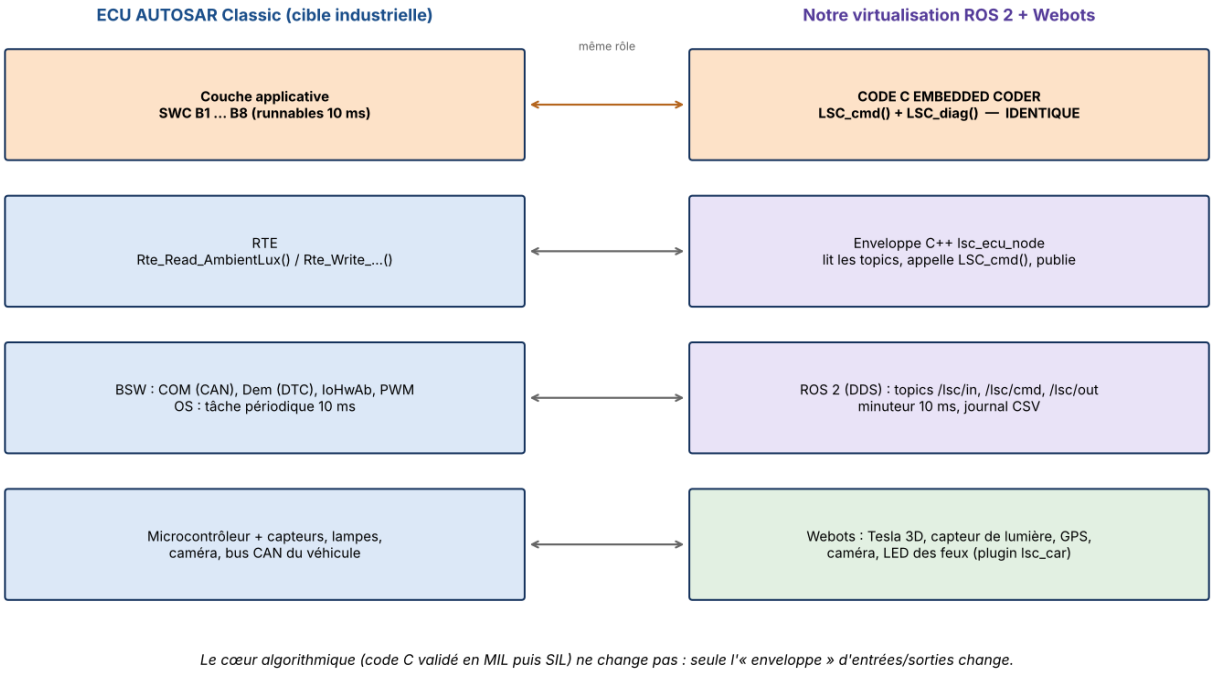


Figure 1 — Couches d'un calculateur AUTOSAR Classic et leurs équivalents dans notre virtualisation

Idée clé pour le jury : le code de l'éclairage est indépendant de ce qui l'entoure. On l'a écrit et validé une seule fois (MIL puis SIL) ; ensuite on l'a « branché » sur ROS 2 en écrivant seulement une enveloppe d'entrées/sorties. Dans un vrai calculateur, cette enveloppe serait la RTE AUTOSAR. C'est le principe de séparation application / logiciel de base qui fonde AUTOSAR.

2. ROS 2 : les notions indispensables

ROS 2 (Robot Operating System 2) n'est pas un système d'exploitation : c'est un **intergiciel** (middleware) et un ensemble d'outils pour faire communiquer des programmes. Il est très utilisé pour les robots et les prototypes de véhicules autonomes (par exemple la pile open source Autoware). La version utilisée ici est **Lyrical Luth** (2026), associée à Ubuntu 26.04.

Notion	Définition	Dans notre projet
Nœud (node)	Un programme qui participe au système ROS 2	lsc_ecu, lsc_car, lsc_hud, lsc_demo
Topic	Un canal nommé sur lequel circulent des messages	/lsc/in/lux, /lsc/out/duty, /lsc/cmd/ignition…
Message	Le type de donnée transporté (comme une trame CAN)	std_msgs/Float64, Bool, UInt8MultiArray, String
Publisher / Subscriber	Un nœud publie sur un topic ; un ou plusieurs nœuds s'y abonnent	lsc_car publie la luminosité ; lsc_ecu s'y abonne
DDS	Le protocole réseau sous ROS 2 (découverte automatique, qualité de service)	Transporte tous nos topics
Paramètre	Une valeur réglable d'un nœud	tunnel_lux = 150 (plugin lsc_car)
Paquet (package)	L'unité de code : ament_cmake (C/C++) ou ament_python	lsc_ecu (C/C++), lsc_virtual (Python)
Espace de travail	Dossier contenant les paquets, compilé avec colcon	~/ros2_ws (src, build, install)
Fichier de lancement	Démarre plusieurs nœuds d'un coup (launch file Python)	lsc_demo_launch.py

Pourquoi le modèle « publish / subscribe » ressemble au CAN

Sur un bus CAN, un calculateur émet une trame (par exemple la vitesse) sans savoir qui la lit ; tous les calculateurs intéressés la reçoivent. ROS 2 fonctionne de la même façon : **/lsc/in/speed** joue le rôle de la trame de vitesse émise par l'ESP, et le calculateur d'éclairage s'y abonne. La différence : ROS 2 passe par Ethernet/mémoire partagée avec DDS, pas par un bus à arbitrage temps réel.

Les commandes à connaître (terminal Ubuntu)

```
source /opt/ros/lyrical/setup.bash           # charger ROS 2 (fait automatiquement par ~/.bashrc)
cd ~/ros2_ws && source install/local_setup.bash   # charger nos paquets
ros2 node list                                  # nœuds en cours : /lsc_ecu /lsc_car /lsc_hud /lsc_demo
ros2 topic list                                 # tous les topics
ros2 topic echo /lsc/out/light_state            # voir l'état des feux en direct (0..5)
ros2 topic hz /lsc/out/duty                     # vérifier la période : ~100 Hz = 10 ms
ros2 topic pub --once /lsc/cmd/turn_left std_msgs/msg/Bool "{data: true}"   # actionner le clignotant
rqt_graph                                       # dessin du graphe nœuds/topics
colcon build --packages-select lsc_ecu lsc_virtual                      # compiler nos paquets
```

Astuce démonstration : pendant la soutenance, ouvrir un 2e terminal et lancer **ros2 topic echo /lsc/out/ahb_state** : le jury voit les décisions du calculateur défiler pendant que la voiture roule.

3. Webots : le monde virtuel

Webots (Cyberbotics, open source) est un simulateur 3D avec moteur physique. Il tourne sous **Windows** car il a besoin de la carte graphique ; ROS 2 tourne dans **Ubuntu (WSL 2)**. Les deux communiquent par **TCP** (port 1234).

Notion Webots	Explication	Dans notre projet
Monde (.wbt)	Fichier texte qui décrit toute la scène	lsc_demo.wbt : routes, tunnel, maison, arbres, 2 Prius, Tesla
Nœud / PROTO	Objet de la scène ; un PROTO est un modèle réutilisable	TeslaModel3, ToyotaPrius, StraightRoadSegment, CurvedRoadSegment
Robot et contrôleur	Un robot exécute un programme ; « extern » = programme lancé de l'extérieur	La Tesla : controller "" piloté par ROS 2
Device	Capteur ou actionneur d'un robot	LightSensor, GPS, InertialUnit, Camera (recognition), LED
Superviseur	Robot spécial qui peut modifier la scène	lsc_ambiance : nuit, textes à l'écran, conducteur, porte
basicTimeStep	Pas de simulation	10 ms, comme la période des runnables
Viewpoint	La caméra du spectateur	« Mounted Shot » : suit la voiture par l'arrière

Comment Webots parle à ROS 2 : webots_ros2_driver

Le paquet officiel **webots_ros2** (compilé depuis ses sources pour Lyrical) fournit **webots_ros2_driver**. Pour chaque robot, on écrit un petit fichier URDF qui désigne un **plugin Python**. Le pilote charge ce plugin et appelle sa méthode **step()** à chaque pas de simulation (10 ms). Le plugin a accès aux devices du robot *et* à ROS 2 :

```
<!-- resource/lsc_car_demo.urdf -->
<robot name="LSC Tesla demo"><webots>
    <plugin type="lsc_virtual.lsc_demo_plugin.LscDemoPlugin" />
</webots></robot>

class LscDemoPlugin:
    def init(self, webots_node, properties):          # appelé une fois
        self.robot = webots_node.robot                # la Tesla (API Webots « Driver »)
        self.light = self.robot.getDevice('ambient_light'); self.light.enable(10)
        self.node = rclpy.create_node('lsc_car')      # nœud ROS 2 du plugin
        self.pub_lux = self.node.create_publisher(Float64, '/lsc/in/lux', 10)
    def step(self):                                   # appelé toutes les 10 ms
        self.pub_lux.publish(Float64(data=self.light.getValue()))
        ... applique /lsc/out/duty aux LED des feux ...
```

Les capteurs et feux ajoutés à la Tesla

Device Webots	Emplacement	Rôle
LightSensor « ambient_light »	Toit, orienté vers le ciel	Mesure la luminosité réelle de la scène (31 000 lux au soleil, 25 lux la nuit)
GPS + InertialUnit	Centre (essieu arrière)	Position, vitesse, cap : guidage le long de la route
Camera + Recognition	Pare-brise	Détecte les véhicules (objets annotés) à moins de 90 m
LED + SpotLight	Avant	Faisceaux de croisement (court) et de route (long)
LED	Phares, coins avant et arrière	DRL, clignotants avant/arrière, feux arrière (position / stop / ESS)

4. Notre projet : paquets, nœuds et topics

```
~/ros2_ws/src/
├── webots_ros2/                (officiel, compilé depuis les sources)
├── lsc_ecu/                    (ament_cmake : LE CALCULATEUR)
│   ├── ecu/*.c, *.h            code C généré par Embedded Coder (B1..B8) — identique au SIL
│   └── src/lsc_ecu_node.cpp    enveloppe ROS 2 : topics <-> LSC_cmd() / LSC_diag()
└── lsc_virtual/                (ament_python : LE VÉHICULE VIRTUEL)
    ├── lsc_virtual/lsc_demo_plugin.py   Tesla : capteurs -> ROS 2, ROS 2 -> feux, guidage
    ├── lsc_virtual/lsc_demo_hud.py      superviseur : nuit, textes à l'écran, conducteur, maison
    ├── lsc_virtual/demo.py              conducteur automatique + 15 vérifications PASS/FAIL
    ├── worlds/lsc_demo.wbt              monde : virages, tunnel, maison, 2 voitures en face
    └── launch/lsc_demo_launch.py        lance tout (Webots + 4 nœuds)
```

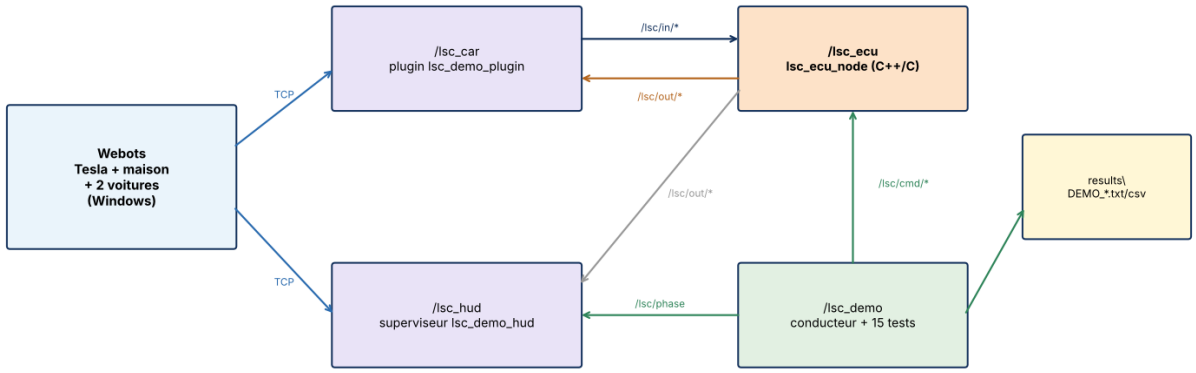


Figure 2 — Graphe ROS 2 de la démonstration (équivalent de rqt_graph)

Topic	Type	Sens	Contenu
/lsc/in/lux	Float64	Webots → ECU	Luminosité mesurée (lux) — 150 lux imposés dans le tunnel (éclairage du tunnel)
/lsc/in/speed, /lsc/in/decel	Float64	Webots → ECU	Vitesse (km/h, GPS) et décélération (m/s²)
/lsc/in/vehicle_detected	Bool	Webots → ECU	Véhicule en face vu par la caméra
/lsc/cmd/light_switch	Int32	Conducteur → ECU	Commodo : 0 OFF, 1 AUTO, 2 POSITION, 3 CROISEMENT
/lsc/cmd/turn_left, turn_right, hazard, flash	Bool	Conducteur → ECU	Levier de clignotant, warning, appel de phares
/lsc/cmd/ignition, door_closed, brake	Bool	Véhicule → ECU	Contact, porte, pédale de frein
/lsc/cmd/lamp_fail, cam_fault, sensor_fault	Int32 / Bool	Injection de pannes	Ampoule grillée, caméra ou capteur en défaut
/lsc/out/duty	UInt8MultiArray	ECU → Webots	8 PWM (%) : croisement G/D, route, DRL, clignotants G/D, arrière, antibrouillard
/lsc/out/light_state, ahb_state	UInt8	ECU → affichage	État des feux (0..5) et de l'AHB (0..4)
/lsc/out/hazard, ess, fmh, dtc	Bool / UInt16	ECU → affichage	Détresse, ESS, follow-me-home, défauts mémorisés

Un cycle de 10 ms, pas à pas

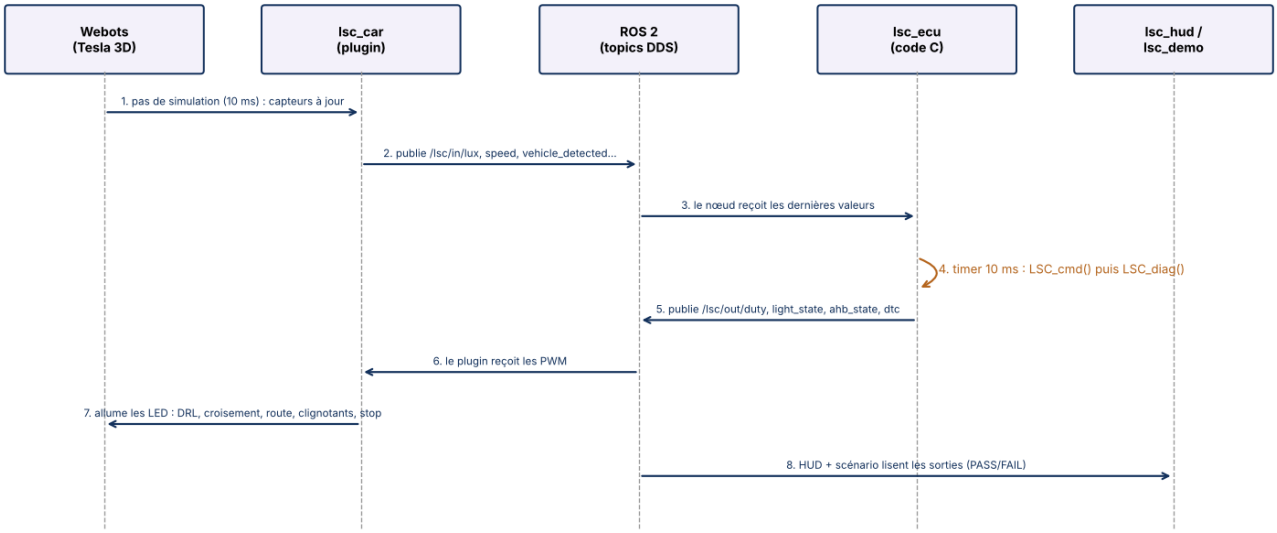


Figure 3 — Ce qui se passe à chaque période de 10 ms

Le cœur du nœud lsc_ecu (extrait)

```
extern "C" {                        // le code généré est du C pur
#include "LSC_cmd.h"
#include "LSC_diag.h" }

void step() {                       // appelé par un minuteur toutes les 10 ms
  // Runnable 1 : B1 -> B2 -> B3 -> B4/B8 -> B5 -> B7
  LSC_cmd(ign_, sw_, flash_, ..., lux_, speed_, decel_, veh, cam, door_, vbat_,
          fault_prev_, deg_prev_,                  // valeurs du pas précédent (comme la RTE)
          duty, &veff, dcmd, &lst, &fmh, &ahb, &haz, &ess, &luxf);
  Plant_Lamps(duty, vbat_, lf, cur);               // courant des ampoules (modèle)
  // Runnable 2 : B6 diagnostic
  LSC_diag(dcmd, cur, false, fa, &dtc, &deg, &wn);
  duty_pub_->publish(...);  state_pub_->publish(...);  ...
}
```

5. Le lien avec AUTOSAR

5.1 Rappel : AUTOSAR Classic en 5 notions

Notion AUTOSAR	Définition
SWC (Software Component)	Brique logicielle applicative, indépendante du matériel
Port (Sender/Receiver, Client/Server)	Point d'entrée/sortie d'un SWC ; transporte des « data elements » ou des appels de service
Runnable	Fonction d'un SWC appelée par le système d'exploitation (par exemple toutes les 10 ms)
RTE (Runtime Environment)	Code généré qui relie les ports des SWC entre eux et au logiciel de base (Rte_Read / Rte_Write)
BSW (Basic Software)	Logiciel de base standard : OS, COM (CAN), Dem (défauts), NvM (mémoire), IoHwAb, drivers
VFB / ARXML	Vue « bus virtuel » des échanges entre SWC, décrite dans des fichiers XML (ARXML)

5.2 Les SWC du cahier des charges et nos blocs

Le cahier des charges (section 6.2) découpe la fonction en 8 SWC. Dans MATLAB, nous avons développé 8 blocs (B1 à B8). La correspondance est la suivante :

SWC (cahier des charges)	Runnable (période)	Bloc(s) réalisé(s)	Fonction C générée
SwitchInput_SWC	Run_SwitchRead (10 ms)	B1 (commodo, anti-rebond)	B1_Acquisition()
LightSensor_SWC	Run_LuxProcess (10 ms)	B1 (filtre, défaut capteur)	B1_Acquisition()
CameraAHB_SWC	Run_ObjectEval (20 ms)	B3 (feux de route automatiques)	AHB_Logic()
LightingMgr_SWC	Run_LightingMgr (10 ms)	B2 + B4 + B8 (auto, arbitrage, follow-me-home)	AutoHeadlampLogic(), B4_Manager()
TurnIndicator_SWC	Run_Flasher (10 ms)	B5 (clignotants, warning, hyper)	B5_Signal()
BrakeLight_SWC	Run_BrakeLight (10 ms)	B5 (feux stop, ESS)	B5_Signal()
LampDiag_SWC	Run_LampDiag (100 ms)	B6 (diagnostic, DTC)	B6_Diag()
LampActuator_SWC	Run_LampOut (10 ms)	B7 (PWM, régulation, soft-start)	B7_Driver()

Écart à expliquer honnêtement : dans notre réalisation, deux runnables regroupent les blocs (**LSC_cmd** et **LSC_diag**), tous deux à 10 ms. Le cahier des charges prévoit CameraAHB à 20 ms et LampDiag à 100 ms : dans un vrai projet AUTOSAR, la RTE et l'OS appelleraient chaque runnable à sa propre période ; le code de chaque bloc ne change pas.

5.3 Les ports AUTOSAR et les topics ROS 2

Port / data element (cahier des charges)	Type AUTOSAR	Topic ROS 2 équivalent
LightSwitchPos	Sender/Receiver	/lsc/cmd/light_switch
AmbientLux / LuxValid	Sender/Receiver	/lsc/in/lux (+ /lsc/cmd/sensor_fault)
RoadFree / CamStatus	Sender/Receiver	/lsc/in/vehicle_detected, /lsc/in/cam_ok (+ cam_fault)
VehicleSpeed	Sender/Receiver (Com, CAN ESP)	/lsc/in/speed
LongAccel	Sender/Receiver (Com, CAN ESP)	/lsc/in/decel
IgnitionStatus / DoorStatus / BatteryVoltage	Sender/Receiver (Com)	/lsc/cmd/ignition, /lsc/cmd/door_closed, /lsc/in/vbat
LampRequest	Sender/Receiver	/lsc/out/duty (8 PWM)
LampFault	Sender/Receiver	interne (fault_prev) + /lsc/out/dtc
DiagEvent	Client/Server → Dem	/lsc/out/dtc (bitfield des défauts)
NvCalib	Client/Server → NvM	constantes LSC_params_const (figées dans le code)

5.4 Même cœur, deux enveloppes : RTE AUTOSAR ou ROS 2

Voici, côte à côte, comment le même code C serait appelé dans un calculateur AUTOSAR et comment il est appelé dans notre nœud ROS 2. Seules les lignes d'entrée/sortie changent :

```
/* ---- Dans un calculateur AUTOSAR (runnable appelé par l'OS toutes les 10 ms) ---- */
void Run_LightingMgr(void) {
    uint16 lux;   Rte_Read_AmbientLux_Value(&lux);        /* port Sender/Receiver */
    uint16 v;     Rte_Read_VehicleSpeed_Value(&v);        /* signal CAN via COM   */
    ...
    LSC_cmd(..., (double)lux, ..., v * 0.01, ..., duty, ...);   /* MÊME FONCTION */
    Rte_Write_LampRequest_Duty(duty);                     /* vers LampActuator    */
}

/* ---- Dans notre nœud ROS 2 (minuteur de 10 ms) ---- */
void LscEcuNode::step() {
    /* lux_, speed_ ... ont été mis à jour par les abonnements aux topics /lsc/in/* */
    LSC_cmd(..., lux_, ..., speed_, ..., duty, ...);      /* MÊME FONCTION */
    duty_pub_->publish(duty);                             /* topic /lsc/out/duty */
}
```

C'est le patron de conception « **adaptateur** » : le cœur algorithmique ne connaît ni la RTE ni ROS 2. C'est exactement ce qu'AUTOSAR cherche à obtenir avec la séparation **SWC / RTE / BSW** : un SWC est réutilisable sur un autre calculateur ou un autre réseau sans être réécrit.

5.5 Correspondance complète des concepts

AUTOSAR Classic	Notre réalisation (code C)	Notre virtualisation ROS 2 / Webots
SWC B1..B8	Fonctions C générées par Embedded Coder	Dans le nœud lsc_ecu
Runnable 10 ms	LSC_cmd(), LSC_diag()	Minuteur ROS 2 de 10 ms
Port Sender/Receiver	Arguments des fonctions	Topics /lsc/in, /lsc/cmd, /lsc/out
RTE (Rte_Read / Rte_Write)	Banc de test SIL (sil_harness.c)	Enveloppe C++ lsc_ecu_node + DDS
Échange entre runnables via RTE	fault_prev / degraded_prev (pas précédent)	Variables membres du nœud
COM + bus CAN	—	DDS (réseau ROS 2)
Dem (gestion des défauts)	B6 : bitfield DTC	/lsc/out/dtc, affiché à l'écran
IoHwAb + driver PWM	B7 : rapports cycliques	Plugin lsc_car : LED et projecteurs 3D
OS (tâche périodique)	—	Minuteur 10 ms (Linux non temps réel)
ARXML	Non généré (Simulink Coder absent)	package.xml + fichiers de lancement (description du système)

5.6 Et AUTOSAR Adaptive ?

AUTOSAR existe en deux variantes. **Classic** (celle du cahier des charges) vise les calculateurs temps réel sur microcontrôleur, comme l'éclairage. **Adaptive** vise les calculateurs puissants (conduite automatisée, passerelles) ; sa couche de communication orientée services (ara::com) peut s'appuyer sur SOME/IP ou sur **DDS** — le même protocole que ROS 2. ROS 2 est donc très utilisé pour **prototyper** des fonctions avant leur intégration AUTOSAR.

5.7 Le chemin vers un vrai calculateur AUTOSAR

1. Installer Simulink Coder (licence) : AUTOSAR Blockset génère alors le code avec les appels Rte_Read/Rte_Write et le fichier **ARXML** de chaque SWC.
2. Importer l'ARXML dans un outil d'intégration (configuration RTE + BSW : COM, Dem, NvM, OS).
3. Compiler pour le microcontrôleur cible et rejouer les mêmes scénarios en **HIL** (le simulateur envoie les signaux CAN).

4. Comparer aux résultats MIL / SIL / virtualisation : mêmes seuils, mêmes temps (0,3 s tunnel, 2 s AHB, 30 s follow-me-home).

6. La démonstration complète

Un seul lancement joue tout le trajet. Le nœud **lsc_demo** joue le rôle du conducteur (clignotants, accélération, freinage, contact, porte) et injecte une panne d'ampoule ; le superviseur affiche en haut de l'écran la phase, la vitesse, l'état des feux et l'exigence vérifiée.



Virage à droite, accélération forte, détresse automatique après l'ESS, véhicule en face, feux de route, follow-me-home



Follow-me-home : le conducteur marche dans le faisceau des phares, entre ; la maison s'allume, les feux s'éteignent

Test	Exigence	Vérification	Mesuré
D-01	REQ-LSC-05	Départ de jour : DRL seuls	DRL

Test	Exigence	Vérification	Mesuré
D-02	REQ-LSC-16	Virage à droite : clignotant à 1,5 Hz	5 à 7 impulsions / 4 s
D-03 / D-04	REQ-LSC-08 / 07	Tunnel : croisement ≤ 0,4 s ; DRL après 5 s	0,31–0,32 s / 5,02–5,04 s
D-05 / D-06	REQ-LSC-23 / B6	Ampoule grillée : hyper-clignotement ; DTC	12 impulsions / 4 s ; DTC mémorisé
D-07	REQ-LSC-20	Freinage d'urgence à 85 km/h : ESS	11 éclats des feux stop
D-08 / D-09	REQ-LSC-21	Détresse automatique à l'arrêt, coupée au redémarrage	PASS
D-10 / D-11	REQ-LSC-06 / 10-11	Crépuscule : croisement ; nuit : feux de route	PASS
D-12 / D-13	REQ-LSC-13 / 12	Véhicule en face : route coupée ; rallumée après 2 s	PASS ; réarmement 2,0 s
D-14 / D-15	REQ-LSC-25	Follow-me-home ; extinction 30 s après la porte	30,0 s

Résultat : 15 / 15 PASS (fichier results\DEMO_resultats.txt). Avec le trajet d'essai (17 tests), la virtualisation totalise 32 tests réussis.

7. Mode opératoire

7.1 Lancer la démonstration

1. Menu Démarrer → **Ubuntu** (ou **wsl -d Ubuntu** dans PowerShell). Attention : pas « Ubuntu 24.04 ».
2. Taper : **cd ~/ros2_ws && source install/local_setup.bash**
3. Taper : **ros2 launch lsc_virtual lsc_demo_launch.py** — Webots s'ouvre sous Windows (environ 20 s).
4. Ne rien toucher pendant environ 3 min 30 : le trajet se déroule, puis « FIN DE LA DÉMONSTRATION — 15 / 15 tests PASS ».
5. Pour arrêter : Ctrl + C dans le terminal, puis fermer Webots.

Variante	Commande
Route libre, sans conducteur automatique	ros2 launch lsc_virtual lsc_demo_launch.py auto:=false
Trajet d'essai rectiligne + 17 tests	ros2 launch lsc_virtual lsc_launch.py scenario:=true
Tableau de bord texte (2e terminal)	ros2 run lsc_virtual dashboard

7.2 Après une modification du modèle MATLAB

1. Modifier le bloc dans scripts\Bx_logic.txt, relancer ses tests MIL (run_bX) et l'intégration (run_int, run_int_scenarios).
2. Dans MATLAB : **sil_all** — régénère le code C dans sil\codegen\lib\LSC_ECU et les fichiers de référence.
3. Rejouer le SIL (sil_harness.c) : 0 écart attendu.
4. Copier les fichiers .c/.h générés dans **lsc_ecu/ecu/**, puis **colcon build --packages-select lsc_ecu**.
5. Relancer la démonstration : le véhicule virtuel utilise immédiatement le nouveau code.

Cette chaîne garantit la **traçabilité** : le code qui tourne dans Webots est exactement celui testé en SIL, lui-même issu du modèle testé en MIL.

8. Dépannage

Symptôme	Cause	Solution
« ros2 : command not found » ou ros2_ws introuvable	Mauvais Ubuntu (24.04 au lieu de 26.04)	Ouvrir « Ubuntu », ou wsl -d Ubuntu
Webots reste à 0:00:00	Un contrôleur extern n'est pas connecté	Vérifier dans le terminal la ligne « connected » des 3 robots
La voiture ne bouge pas	Le plugin a planté (voir Traceback)	Lire le terminal ; relancer
Pas de feux de route la nuit	Luminosité ≥ 50 lux ou vitesse < 40 km/h	Lire les valeurs affichées en haut de l'écran
Avertissements « R2025b … forward compatibility »	Cache Webots	Sans effet, à ignorer

Limites de la virtualisation

Limite	Explication
Temps réel	Minuteur de 10 ms sous Linux non temps réel ; Webots tourne à ~0,95× le temps réel
Capteurs idéalisés	Le capteur de lumière ne voit que les lumières de Webots ; 150 lux imposés dans le tunnel
Caméra	Détection par la « recognition » de Webots (objets annotés), pas par un réseau de neurones
Réseau	Topics ROS 2 au lieu de trames CAN ; pas de RTE AUTOSAR générée
Conducteur	Guidage par GPS sur une trajectoire connue, pas un vrai pilote

Questions probables du jury

- **Pourquoi ROS 2 dans un projet AUTOSAR ?**
  ROS 2 sert ici de banc de virtualisation : il relie le calculateur à un véhicule 3D. Le cœur reste le code C AUTOSAR-compatible généré depuis le modèle ; l'enveloppe ROS 2 joue le rôle de la RTE.
- **Le code est-il vraiment le même qu'en SIL ?**
  Oui, les fichiers .c/.h générés par Embedded Coder sont copiés sans modification dans lsc_ecu/ecu. Seul lsc_ecu_node.cpp est ajouté.
- **Où est la RTE dans votre virtualisation ?**
  Le nœud lsc_ecu en tient le rôle : il lit les entrées (topics), appelle les runnables dans l'ordre et publie les sorties, et transmet au pas suivant les données échangées entre runnables (défauts lampes).
- **Quelle différence entre un topic ROS 2 et un port AUTOSAR ?**
  Les deux sont des échanges émetteur/récepteur typés. Un port AUTOSAR est configuré statiquement (ARXML) et implémenté par la RTE générée ; un topic ROS 2 est découvert dynamiquement par DDS à l'exécution.
- **Comment passeriez-vous à un vrai calculateur ?**
  Génération AUTOSAR (Simulink Coder + AUTOSAR Blockset) avec ARXML et appels Rte_*, intégration BSW, compilation pour la cible, puis HIL avec les mêmes scénarios.
- **Comment sont détectés les véhicules en face ?**
  Par la caméra de Webots avec la fonction recognition, qui renvoie les objets visibles et leur position ; un véhicule à moins de 90 m met vehicle_detected à 1 (port RoadFree = 0).
- **Pourquoi le follow-me-home s'éteint-il après 30 s ?**
  REQ-LSC-25/29 : la temporisation démarre à la fermeture de la dernière porte, laisse le temps de rejoindre le domicile, puis coupe pour préserver la batterie.

Glossaire

Terme	Signification
AHB	Auto High Beam : feux de route automatiques
DDS	Data Distribution Service : protocole sous ROS 2
DRL	Daytime Running Lights : feux de jour
DTC	Diagnostic Trouble Code : défaut mémorisé
ESS	Emergency Stop Signal : feux stop clignotants au freinage d'urgence
FMH	Follow-me-home : éclairage d'accompagnement
MIL / SIL / HIL	Model / Software / Hardware-In-the-Loop
RTE	Runtime Environment (AUTOSAR)
SWC	Software Component (AUTOSAR)
WSL	Windows Subsystem for Linux